SwarRakshak: Full-Stack React Implementation and Deep Audit Prompt

Copy and use the prompt below with a coding agent or senior developer.

Role

Act as a **senior React and Python backend developer, software architect, database engineer, security reviewer, QA engineer, and code auditor**. You are working with a beginner hackathon team on **SwarRakshak**.

You must implement, review, test, audit, debug, and improve the project rather than stopping at planning. Inspect the existing repository before changing anything. Preserve working behavior, avoid unnecessary rewrites, and explain important changes in beginner-friendly language.

The project is a hackathon prototype, not a production banking system. Keep the implementation small enough to finish and demonstrate reliably.

Product identity

Product name: SwarRakshak

One-line description: SwarRakshak is a privacy-aware voice-risk assistant that analyzes call audio and request context to warn users about possible voice-cloning impersonation before a sensitive action is approved.

Core product promise:

> When a voice can no longer be trusted by itself, SwarRakshak combines voice authenticity, speaker consistency, and request context to recommend the safest next action before fraud occurs.

Product problem

Artificial intelligence can generate a voice that sounds like a real executive, customer, government officer, or family member. Attackers may use cloned voices and leaked information to request fund transfers, account changes, passwords, confidential documents, or urgent approvals.

Caller ID can be spoofed. Familiarity with a voice is no longer sufficient. Employees under pressure may skip normal verification. Existing defenses such as callbacks, multifactor

authentication, fraud rules, and voice biometrics may not provide a clear warning at the moment a risky decision is being made.

SwarRakshak must help the user answer:

1. Does the audio contain patterns associated with synthetic or manipulated speech?
2. Does the current voice resemble a trusted reference sample, when one is available?
3. Is the request suspicious because of its context, urgency, caller origin, or transaction value?

The system must provide a risk warning and recommended verification action. It must not claim to prove that a caller is fake or genuine.

Target user and main persona

The main MVP user is a bank or company finance employee who receives voice instructions related to money transfers or sensitive account actions.

Example persona:

- **Name:** Priya
- **Role:** Finance Operations Executive
- **Situation:** A caller claiming to be the CFO requests an urgent transfer to a new account.
- **Difficulty:** The caller sounds convincing and the existing workflow does not provide an audio-risk warning.
- **Desired outcome:** Priya receives a clear explanation and knows to verify the request using a trusted channel instead of trusting the caller’s number.

Secondary users include fraud analysts, call-center agents, executives, government employees, and future telecom or enterprise communication providers.

MVP product behavior

Build a web-capable React frontend and a Python FastAPI backend.

The core user flow is:

1. User starts a new analysis.
2. User uploads or records a short current-call audio sample.
3. User optionally uploads a consented reference voice sample.
4. User enters call context.

5. Backend validates and temporarily stores the audio outside the public directory.
6. Backend converts, validates, and segments the audio.
7. Voice-authenticity analysis estimates whether the audio may be synthetic or manipulated.
8. Optional speaker comparison checks similarity with the reference sample.
9. Context rules add risk contributions.
10. Risk engine calculates a transparent score from 0 to 100.
11. Backend returns risk level, confidence, reasons, segment results, and recommended action.
12. Frontend displays the result in understandable language.
13. Backend deletes raw audio by default and stores only metadata and reason codes.

The MVP may simulate near-real-time analysis by processing short sequential segments. It does not need mobile-carrier, telecom, VoIP, banking-core, or real-money integration.

Three required demo scenarios

Scenario 1: Genuine normal call

- Known caller number.
- Normal request.
- Normal transaction value.
- Normal procedure.
- Voice is consistent with the reference sample.
- Expected result: Low risk or a cautious normal result.

Scenario 2: Possible cloned voice

- Unknown caller number.
- Urgent request.
- High-value transaction.
- Procedure bypass requested.
- Synthetic-voice evidence is high.
- Reference voice comparison is inconsistent.
- Expected result: High risk.

- Recommended action: Pause the transfer and call the person using an independently verified number.

Scenario 3: Natural voice but suspicious context

- Voice appears natural.
- Caller requests a high-value transfer urgently.
- Caller asks to bypass a second approval.
- Expected result: Medium or High risk based on configured thresholds.
- This proves that SwarRakshak does not rely only on deepfake detection.

Also implement an inconclusive case for short, silent, corrupted, or noisy audio. Never invent a confident result when evidence is insufficient.

Risk scoring rules

Use a transparent weighted scoring engine for the hackathon:

Signal	Maximum contribution
Synthetic-voice indication	40 points
Speaker mismatch	30 points
Unknown caller number	10 points
High-value request	10 points
Urgent request or procedure bypass	10 points

Cap the score at 100.

Initial risk bands:

- **Low:** 0–29
- **Medium:** 30–59
- **High:** 60–100
- **Inconclusive:** evidence is insufficient, regardless of numerical score

These are prototype settings, not scientifically validated thresholds. Store the scoring-rules version with every result.

Required reason codes may include:

- `SYNTHETIC_AUDIO_PATTERN`
- `REFERENCE_VOICE_MISMATCH`
- `UNKNOWN_CALLER_NUMBER`
- `HIGH_VALUE_REQUEST`
- `URGENT_OR_BYPASS_REQUEST`
- `INSUFFICIENT_AUDIO_EVIDENCE`

Every flagged result should show at least two human-readable reasons when evidence allows.

Recommended actions:

- Low: Continue normal verification procedures; do not rely on voice alone for sensitive actions.
- Medium: Do not share sensitive information yet; ask a verification question and confirm through a trusted channel.
- High: Pause the action; independently call the person and require second-person approval or multifactor authentication.
- Inconclusive: Treat the request as unverified and use an independent verification method.

React frontend requirements

Use **React and JavaScript** as the frontend technology. Build a simple, accessible, web-capable interface.

Required screens

1. **Home screen**
    - Explain the purpose of SwarRakshak.
    - Include a clear “Start analysis” action.
    - State that the system provides a warning, not proof of identity.
2. **Audio input screen**
    - Pick or record current-call audio.

- Optionally pick a reference voice sample.
- Show file name, duration, validation state, and privacy message.
- Do not upload until required input is valid.

3. **Context form screen**
   - Caller number status: known, unknown, hidden.
   - Request urgency: normal, urgent.
   - Transaction value: low, medium, high.
   - Procedure status: normal, bypass requested.
   - Caller relationship: customer, vendor, executive, unknown.
   - Use accessible labels, validation messages, and keyboard navigation.

4. **Analysis progress screen**
   - Show upload, preprocessing, audio analysis, speaker comparison, and scoring status.
   - Show errors safely without stack traces.

5. **Result screen**
   - Show risk score, risk level, confidence, evidence sufficiency, reasons, recommended action, and segment timeline.
   - Do not use color as the only risk indicator.
   - For High risk, make the verification recommendation prominent.
   - For Inconclusive, explain that the evidence is insufficient.

6. **Analysis summary screen**
   - Show metadata-only flagged events for a fraud analyst.
   - Do not show raw audio or full transcripts.

React implementation expectations

- Use a clear folder structure such as `src/components`, `src/pages`, `src/services`, `src/models`, and `src/state`.
- Use typed JavaScript models or TypeScript interfaces for API responses, according to the existing repository setup.
- Centralize API calls in a service class.
- Handle loading, success, validation error, server error, timeout, and empty states.

- Use `FormData` and multipart upload for audio files.
- Configure API base URL through environment or build configuration, not hardcoded production URLs.
- Avoid putting secrets in the React application.
- Add unit tests for model parsing, validation, score display mapping, and error handling.

Backend requirements

Use Python with FastAPI. Keep the backend as a modular monolith for the hackathon.

Recommended modules:

- API routes.
- Request and response schemas.
- Audio validation and preprocessing.
- Voice authenticity model adapter.
- Optional speaker comparison adapter.
- Context and policy rules.
- Risk scoring engine.
- Recommendation engine.
- Database repository layer.
- Privacy and temporary-file cleanup.
- Health checks.

Do not spread SQL throughout route handlers. Keep database access in a repository or database module.

Audio handling

- Support WAV, MP3, and M4A where dependencies allow.
- Validate MIME type, decoded format, file size, and duration.
- Suggested maximum file size: 25 MB.
- Suggested maximum duration: 60 seconds per file.
- Convert to the model’s required format, such as mono 16 kHz WAV.
- Split current audio into two-to-five-second segments.

- Reject empty, corrupted, silent, or unusable audio.
- Use synthetic or consented samples only.
- Store raw audio temporarily outside the public web folder.
- Delete raw audio after processing by default.

If a reliable model is not available, use a clearly labeled prototype classifier or prepared demo result. Never claim production-level accuracy.

Required REST API

Base path:

```
/api/v1
```

Implement only these four endpoints for the MVP:

1. Health check

```
GET /api/v1/health
```

Authentication: none in local demo mode.

Success response:

```
{
  "status": "ok",
  "service": "swarrakshak-api",
  "version": "0.1.0",
  "audio_model": "ready",
  "speaker_model": "ready",
  "checked_at": "2026-09-16T14:30:00Z"
}
```

Return `200` when usable and `503` when required analysis dependencies are unavailable.

2. Create analysis

```
POST /api/v1/analyses
Content-Type: multipart/form-data
```

Authentication: optional for local demo, required in deployed mode.

Required form parts:

- `current_audio`: file.
- `caller_number_status`: `known`, `unknown`, or `hidden`.
- `request_urgency`: `normal` or `urgent`.
- `transaction_value`: `low`, `medium`, or `high`.
- `procedure_status`: `normal` or `bypass_requested`.
- `caller_relationship`: `customer`, `vendor`, `executive`, or `unknown`.

Optional form parts:

- `reference_audio`: consented reference file.
- `retain_audio`: defaults to `false`; raw audio retention is discouraged.

Success response must include:

- `event_id`
- `status`
- `created_at`
- `completed_at`
- `risk_score`
- `risk_level`
- `confidence_level`
- `confidence_score`
- `evidence_sufficient`
- `audio_analysis`
- `speaker_analysis`
- `context`

- `reasons`
- `segments`
- `recommended_action`
- `raw_audio_retained`
- `audio_deleted_at`
- `model_version`
- `rules_version`

Example:

```
{
  "event_id": "evt_01J9SWR123",
  "status": "completed",
  "risk_score": 82,
  "risk_level": "high",
  "confidence_level": "high",
  "confidence_score": 0.78,
  "evidence_sufficient": true,
  "audio_analysis": {
    "available": true,
    "synthetic_probability": 0.86
  },
  "speaker_analysis": {
    "available": true,
    "similarity": 0.28,
    "confidence_score": 0.69
  },
  "reasons": [
    {
      "code": "SYNTHETIC_AUDIO_PATTERN",
      "message": "Audio contains patterns associated with synthetic speecl
      "source": "audio",
      "contribution": 40
    }
  ],
  "recommended_action": "Pause the transfer and call the person using an 
  "raw_audio_retained": false,
  "model_version": "prototype-audio-1",
  "rules_version": "risk-rules-1"
}
```


Return:

- `400` for invalid fields or missing current audio.
- `401` for invalid authentication.
- `403` for insufficient permissions.
- `413` for oversized files.
- `415` for unsupported formats.
- `422` for undecodable or unusable audio.

- `500` for unexpected analysis failure.
- `503` for unavailable analysis models.

3. Get one analysis

```
GET /api/v1/analyses/{event_id}
```

Authentication: required in deployed mode.

A finance user may view their own event. Fraud analysts and admins may view permitted events. Do not return raw audio.

Return `200`, `400`, `401`, `403`, `404`, or `500` as appropriate.

4. List analysis summaries

```
GET /api/v1/analyses
```

Authentication: fraud analyst or admin in deployed mode.

Supported query parameters:

- `risk_level`: low, medium, high, inconclusive.
- `status`: started, processing, completed, failed.
- `created_from`: ISO 8601 timestamp.
- `created_to`: ISO 8601 timestamp.
- `page`: minimum 1, default 1.
- `page_size`: 1–100, default 20.
- `sort`: `created_at_desc` or `risk_score_desc`.

Return metadata-only summaries with pagination. Do not return raw audio or full transcripts.

Common error response

Use this shape for all errors:

```
{
  "error": {
    "code": "VALIDATION_ERROR",
    "message": "One or more request fields are invalid.",
    "details": [
      {
        "field": "request_urgency",
        "message": "Value must be 'normal' or 'urgent'."
      }
    ],
    "request_id": "req_01J9SWR999"
  }
}
```

Never return stack traces, access tokens, database paths, temporary file names, raw audio, passwords, or financial credentials.

Database schema

Use SQLite for the hackathon. PostgreSQL can be considered later.

Do not store raw audio bytes in the database.

Table: `users`

Purpose: optional association between an analysis and the user who created it.

Required columns:

- `user_id TEXT PRIMARY KEY`
- `display_name TEXT NOT NULL`
- `role TEXT NOT NULL`
- `created_at DATETIME NOT NULL`
- `is_active INTEGER NOT NULL`

Optional column:

- `email TEXT UNIQUE`

Roles: `finance_user`, `fraud_analyst`, `admin`.

Table: `analysis_events`

Purpose: central lifecycle record for every analysis attempt.

Columns:

- `event_id TEXT PRIMARY KEY`
- `user_id TEXT` foreign key to `users.user_id`, nullable for demo mode
- `status TEXT NOT NULL`: started, processing, completed, failed
- `model_version TEXT NOT NULL`
- `rules_version TEXT NOT NULL`
- `created_at DATETIME NOT NULL`
- `completed_at DATETIME NULL`
- `raw_audio_retained INTEGER NOT NULL DEFAULT 0`
- `audio_deleted_at DATETIME NULL`
- `failure_code TEXT NULL`

Table: `analysis_context`

Purpose: structured caller and transaction context.

Columns:

- `context_id TEXT PRIMARY KEY`
- `event_id TEXT NOT NULL UNIQUE` foreign key to `analysis_events.event_id`
- `caller_number_status TEXT NOT NULL`: known, unknown, hidden
- `request_urgency TEXT NOT NULL`: normal, urgent
- `transaction_value TEXT NOT NULL`: low, medium, high
- `procedure_status TEXT NOT NULL`: normal, bypass_requested
- `caller_relationship TEXT NOT NULL`: customer, vendor, executive, unknown
- `notes TEXT NULL`, if needed; never use for passwords or full transcripts
- `created_at DATETIME NOT NULL`

Table: `analysis_results`

Purpose: final score and recommendation; zero or one per event.

Columns:

- `result_id TEXT PRIMARY KEY`
- `event_id TEXT NOT NULL UNIQUE` foreign key to `analysis_events.event_id`
- `risk_score INTEGER NOT NULL`, 0–100
- `risk_level TEXT NOT NULL`: low, medium, high, inconclusive
- `confidence_level TEXT NOT NULL`: low, medium, high
- `confidence_score REAL NULL`, 0.0–1.0
- `evidence_sufficient INTEGER NOT NULL`
- `recommended_action TEXT NOT NULL`
- `created_at DATETIME NOT NULL`

Table: `analysis_reasons`

Purpose: multiple explanations and score contributions per event.

Columns:

- `reason_id TEXT PRIMARY KEY`
- `event_id TEXT NOT NULL` foreign key to `analysis_events.event_id`
- `reason_code TEXT NOT NULL`
- `message TEXT NOT NULL`
- `contribution INTEGER NOT NULL DEFAULT 0`
- `source TEXT NOT NULL`: audio, speaker, context, system
- `display_order INTEGER NOT NULL`

Table: `analysis_segments`

Purpose: segment-level results for the timeline.

Columns:

- `segment_id TEXT PRIMARY KEY`
- `event_id TEXT NOT NULL` foreign key to `analysis_events.event_id`
- `segment_number INTEGER NOT NULL`
- `start_seconds REAL NOT NULL`
- `end_seconds REAL NOT NULL`
- `synthetic_probability REAL NULL`, 0.0–1.0
- `confidence_score REAL NULL`, 0.0–1.0

- `status TEXT NOT NULL`: analyzed, inconclusive, failed
- `created_at DATETIME NOT NULL`
- Unique constraint on `(event_id, segment_number)`

Relationships

- One user creates many analysis events.
- One analysis event has one context record for a completed analysis.
- One analysis event has zero or one result.
- One analysis event has many reasons.
- One analysis event has many segments.

Required indexes

- `analysis_events.created_at`
- `analysis_events.status`
- `analysis_events.user_id`
- `analysis_results.risk_level`
- `analysis_reasons.event_id`
- `analysis_reasons.reason_code`
- `(analysis_segments.event_id, analysis_segments.segment_number)`

Use foreign keys with cascading child deletion where appropriate. Never delete an event merely because raw audio was deleted.

Architecture and deployment

Use a modular monolith for the hackathon:

```
React frontend
       |
       | HTTPS or localhost HTTP during development
       v
FastAPI backend
       |
       +──> file validation and privacy controller
       +──> audio preprocessing
       +──> voice authenticity adapter
       +──> optional speaker comparison adapter
       +──> context and policy rules
       +──> risk scoring engine
       +──> recommendation engine
       +──> SQLite repository
       +──> metadata-only audit log
```

Hackathon deployment can be a single application environment with:

- React web frontend.
- FastAPI backend.
- SQLite metadata database.
- Local or hosted inference model.
- Private temporary audio directory.

Production deployment is out of scope, but the code should keep modules separated so workers, inference services, authentication, monitoring, and PostgreSQL can be added later.

Deep audit and bug-fixing instructions

After implementation, perform a deep audit. Do not merely state that the code looks correct.

Repository audit

- Inspect the complete repository tree.
- Confirm the actual frontend is React/Dart and the backend is the documented implementation.
- Identify stale React, Node, or incompatible setup instructions.

- Identify dead files, duplicate implementations, unused dependencies, and inconsistent names.
- Verify all imports and package references.
- Check that environment variables are documented and actually used.
- Check that README commands match the repository.

React frontend audit

- Run the project's JavaScript/React lint and type checks, if configured, and fix every actionable warning or error.
- Run the project's frontend test command and fix failing tests.
- Check Android/web build configuration only if those targets exist.
- Test API base URL configuration on local and deployed targets.
- Test file picking and multipart upload.
- Test missing file, invalid file, oversized file, timeout, 400, 401, 403, 404, 413, 415, 422, 500, and 503 responses.
- Check for `setState` after disposal and async lifecycle bugs.
- Check loading states and duplicate-submit prevention.
- Check navigation after widget disposal.
- Check JSON parsing with missing, null, and unexpected fields.
- Check risk labels and colors are accessible and not color-only.
- Check that raw audio is not accidentally cached or logged.
- Check that secrets are not bundled in the app.
- Check browser file-picker behavior and responsive web behavior.

Backend audit

- Run formatting and linting tools available in the repository.
- Run all unit and integration tests.
- Verify FastAPI startup and `/api/v1/health`.
- Validate MIME type and decoded audio, not only file extension.
- Enforce file size and duration limits.
- Keep temporary files outside public directories.

- Delete files in a `finally` cleanup path, including on exceptions.
- Confirm no raw audio bytes enter SQLite.
- Confirm no secrets, tokens, passwords, or full transcripts enter logs.
- Test malformed multipart requests.
- Test missing reference audio and unavailable model behavior.
- Verify insufficient evidence produces `inconclusive`, not false confidence.
- Verify score is capped at 100.
- Verify every score contribution has a reason code.
- Verify event, context, result, reasons, and segments are written transactionally.
- Verify foreign keys and unique constraints.
- Verify unauthorized users cannot read another user's event.
- Verify pagination limits and sorting.
- Avoid returning stack traces or internal file paths.

Database audit

- Apply the SQL schema to a fresh SQLite database.
- Check that all six expected tables exist.
- Check foreign keys, uniqueness, and check constraints.
- Insert the prepared high-risk example data.
- Query the event, context, result, reasons, and timeline.
- Test rollback when one insert fails.
- Test deletion of a parent event and expected child behavior.
- Confirm raw audio columns do not exist.
- Confirm indexes exist and use the expected names.

API contract audit

- Compare frontend request fields with backend request fields.
- Compare backend response models with API documentation.
- Verify field names and enum values exactly.
- Verify all documented endpoints exist.

- Verify all endpoint status codes are realistic.
- Verify common errors use the documented structure.
- Verify `raw_audio_retained` and `audio_deleted_at` reflect actual behavior.
- Verify model and rules versions are stored and returned.

Security and privacy audit

- Confirm default audio retention is false.
- Confirm audio deletion happens on success and failure.
- Confirm uploaded files cannot be executed or served publicly.
- Confirm path traversal is impossible.
- Confirm access tokens are never logged.
- Confirm CORS is not open unnecessarily in deployment.
- Confirm rate limits or basic abuse controls exist where practical.
- Confirm the app never takes an irreversible financial action.
- Confirm user-facing wording does not claim certainty.

Product and UX audit

- Verify the three demo scenarios produce distinguishable outcomes.
- Verify a low score is not described as proof of trust.
- Verify a high score has a clear verification action.
- Verify inconclusive audio is handled safely.
- Verify users understand why a call was flagged.
- Verify the primary flow can be completed by a beginner.

Testing and bug-fix process

Follow this exact sequence:

1. Inspect repository and identify the actual implemented stack.
2. Build or run the project before changing it to capture baseline failures.
3. Create a concise bug list with severity: blocker, high, medium, or low.
4. Fix blockers and high-severity bugs first.

5. Run formatters and static analysis.
6. Run unit, integration, database, and API tests.
7. Test the three demo scenarios.
8. Re-run the deep audit.
9. Check that documentation matches the final implementation.
10. Report remaining limitations honestly.

Do not hide failures by weakening tests, suppressing warnings, or changing requirements without documenting the change.

Required deliverables after implementation

Provide:

1. Working React frontend.
2. Working FastAPI backend.
3. SQLite schema and initialization.
4. API implementation matching this contract.
5. Automated tests for critical logic.
6. Updated README with real commands.
7. Audit report listing bugs found, fixes made, and tests run.
8. Clear list of any remaining limitations.

Final acceptance criteria

The project is acceptable only if:

- React frontend starts successfully.
- Backend starts successfully.
- Health endpoint returns successfully.
- User can submit a valid audio sample and context.
- Backend returns a typed risk result.
- Frontend displays score, level, reasons, confidence, and recommendation.
- At least three demo scenarios can be demonstrated.

- Inconclusive audio is handled safely.
- Raw audio is deleted by default.
- Raw audio is not stored in SQLite.
- API and database contracts match.
- Foreign keys and constraints work.
- Tests and static analysis pass, or remaining failures are explicitly documented.
- No stale product name remains; use **SwarRakshak** consistently.

At the end, summarize:

- Files changed.
- Bugs found.
- Bugs fixed.
- Commands run.
- Tests passed and failed.
- Security and privacy checks performed.
- Remaining risks.
- Exact commands for a beginner to run the final project.

Source documents consolidated by this prompt

This prompt consolidates the project requirements from:

- Product Requirements Document.
- System Architecture.
- Database Design and SQLite schema.
- REST API Specification.

The frontend technology is standardized as **React/JavaScript**, while the backend remains **Python/FastAPI**. If the existing repository differs, audit it first, explain the mismatch, and migrate only when the change is safe and feasible within the hackathon scope.